

Unit 02: Search Techniques

2.1 Uninformed Search Techniques (Blind Search)

Uninformed search techniques, also called **blind search methods**, are search strategies that explore the problem space **without any prior knowledge** about which direction might lead to the goal faster. These algorithms only use the information provided in the **problem definition**—such as the initial state, goal state, and the set of possible actions—without using any additional clues or heuristics.

Common Types of Uninformed (Blind) Search

1. Depth-First Search (DFS)

- **Definition:** Explores a branch of the search tree as far as possible before backtracking.
- **Working:**
 - Starts from the root node.
 - Moves to one child node at a time, going deeper until it reaches a goal or dead end.
 - If dead end is reached, backtracks to explore other branches.
- **Pros:** Uses less memory, simple to implement.
- **Example in Education:** Exploring all possible sequences of learning activities in a module.

2. Breadth-First Search (BFS)

- **Definition:** Explores all nodes at the current depth before moving deeper.
- **Working:**
 - Starts at the root node.
 - Explores all children of a node before exploring grandchildren.
- **Pros:** Guaranteed to find the **shortest path** to the goal.
- **Example in Education:** Searching for a book in a library catalog where shortest path to book location matters.

3. Depth-Limited Search (DLS)

- **Definition:** DFS with a **predefined depth limit** to avoid going infinitely deep.
- **Working:**
 - Similar to DFS but stops expanding nodes beyond a specified depth.
- **Pros:** Prevents infinite loops in deep/infinite search spaces.

- **Example in Education:** Searching only within a certain number of chapters or modules in a course.

4. Iterative Deepening Search (IDS)

- **Definition:** Repeatedly applies DFS with increasing depth limits until the goal is found.
- **Working:**
 - Starts with depth limit = 0.
 - Increases limit incrementally until goal is reached.
- **Pros:** Combines **DFS memory efficiency** with **BFS completeness**.
- **Example in Education:** Exploring learning paths of increasing complexity until student mastery is achieved.

Use in Education: Searching resources in a digital library, exploring paths in learning modules.

2.2 Heuristic Search Techniques (Informed Search)

Heuristic search techniques, also called **informed search methods**, are strategies that use **problem-specific knowledge** (called **heuristics**) to make smarter decisions about which paths to explore first. Unlike uninformed searches, which blindly explore the search space, heuristic searches **estimate the “closeness” of a state to the goal** and prioritize paths that seem more promising. This makes the search process more efficient and faster, especially for large or complex problems.

Common Types of Heuristic (Informed) Search

1. A* Search

- **Definition:** Finds the shortest path to the goal by combining the **actual cost from the start ($g(n)$)** and a **heuristic estimate to the goal ($h(n)$)**.
- **Working:**
 - Evaluates nodes using $f(n) = g(n) + h(n)$.
 - Selects the node with the lowest $f(n)$ value for expansion.
 - Continues until the goal node is reached.
- **Pros:** Guaranteed to find the optimal path if the heuristic is **admissible** (never overestimates).
- **Example in Education:** Choosing the most efficient learning path in an adaptive learning system based on student performance and difficulty of topics.

2. Greedy Best-First Search

- **Definition:** Selects the node that appears **closest to the goal** according to the heuristic value.

- **Working:**
 - Uses only the heuristic function $h(n)$ to prioritize nodes.
 - Does not consider cost so far ($g(n)$).
- **Pros:** Faster than uninformed search in many cases.
- **Example in Education:** AI tutor recommending the next exercise that seems easiest or most relevant to the student.

3. Hill Climbing

- **Definition:** Moves **iteratively in the direction of increasing value**, trying to get closer to the goal.
- **Working:**
 - Evaluates neighboring nodes.
 - Chooses the neighbor with the highest heuristic value.
 - Stops if no better neighbor is found (local maximum).
- **Pros:** Simple and memory-efficient.
- **Example in Education:** Gradually increasing the difficulty of questions for students as they improve, until mastery is achieved.

Use in Education: Optimizing lesson sequencing, adaptive learning paths.

2.3 Adversarial Search (Game/Competition Scenarios)

Adversarial search refers to search strategies used in **competitive environments** where an AI agent must make decisions while **considering the actions of an opponent**. Unlike single-agent search, the goal here is not just to reach a solution but to **maximize the agent's advantage while minimizing the opponent's advantage**. These searches are widely used in games, simulations, and scenarios involving multiple decision-makers.

Major Techniques in Adversarial Search

1. Minimax Algorithm

- **Definition:** A decision-making algorithm used in **competitive (two-player) environments**. The AI tries to **maximize its own advantage** while assuming the opponent will **minimize the AI's gain**.
- **Working:**
 - Builds a **game tree** with all possible moves for both players.
 - Assigns **scores to terminal nodes** (win, lose, draw).
 - Backpropagates scores:
 - AI selects the move with the **maximum score**.

- Opponent is assumed to select moves with the **minimum score** for AI.
- Continues until the best move for AI is found.
- **Pros:** Guarantees optimal decision if the opponent plays optimally.
- **Example in Education:** AI player in educational games (like chess puzzles or math competitions) deciding the best move against a student.

2. Alpha-Beta Pruning

- **Definition:** An optimization technique for the minimax algorithm that **eliminates branches** of the game tree that **cannot affect the final decision**.
- **Working:**
 - Keeps track of two values: **alpha (best value for maximizer so far)** and **beta (best value for minimizer so far)**.
 - If a branch cannot improve the outcome, it is **pruned (skipped)**.
- **Pros:** Reduces the number of nodes evaluated, improving efficiency without changing the result.
- **Example in Education:** Accelerating AI decision-making in educational strategy games without evaluating all possible moves.

Use in Education: AI for educational games, simulations, or quizzes with multiple players.